

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Дальневосточный федеральный университет»
(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Департамент математики и компьютерного моделирования

Отчет
к лабораторной работе №6 по дисциплине «Суперкомпьютеры и параллельное программирование»

По теме: «Передача данных потокам»

Вариант 20

Выполнил: студент группы Б9122-01.03.02мкт

_____ Чирич И.С.
подпись *Ф.И.О.*

Проверил: Доцент

_____ Селезнев Т.Э.
подпись *Ф.И.О*

г. Владивосток
2025

Содержание

Постановка задачи **3**

Индивидуальное задание **4**
Написание программного кода 4
Тестирование 6

Вопросы для защиты работы **8**

Выводы **9**

Постановка задачи

Целью данной лабораторной работы является изучение возможностей передачи аргументов в параллельно выполняемые потоки.

Задачи лабораторной работы:

- Изучить механизм передачи параметров в поток
- Научиться решать практические задачи с использованием потоков с параметрами
- Изучить возможные методы передачи параметров

Индивидуальное задание

Написание программного кода

Создадим шаблон консольного проекта при помощи утилиты `dotnet`:

```
dotnet new console -n thread_args
```

В корне проекта создастся файл Project.cs, в котором и будет написан код лабораторной работы.

Создаем класс Program с методом Main:

```
class Program
{
        static void Main(string[] args)
        {

        }
}
```

Существует несколько способов передачи параметров в поток:

- Передача в конструктор делегата `ParameterizedThreadStart`
- Определение класса, который содержит в себе необходимые данные и метод, выполняемый в потоке

Напишем реализацию индивидуального задания первым способом. Для этого сначала определим тип, который будет передаваться аргументом в делегат конструктора `Thread`:

```
public struct Data
{
        public string message;
        public int offset;
}
```

Структура содержит в себе два параметра, необходимые для реализации функции шифрования: само сообщение и смещение.

Опишем статический метод класса `Program`, который соответствует `ParameterizedThreadStart` и выполняет функцию шифрования:

```
static void EncryptMessage(object o)
{
        string message = ((Data)o).message;
        int n = ((Data)o).offset;
        StringBuilder sb = new StringBuilder();
        foreach (char c in message)
        {
                char current_c = (char)(c + n);
                if(current_c > (char)126)
```

```
                {
                        current_c = (char)(current_c % 127 + ' ');
                }
                sb.Append(current_c);
        }
        Console.WriteLine("Исходная строка: {0}", message);
        Console.WriteLine("Защифрованная строка: {0}", sb);
}
```

Заметим: сигнатура делегата такова, что функция не может возвращать значение и принимает один аргумент типа `object`. С первой проблемой можно бороться, если использовать второй тип передачи аргументов (описанный выше), когда одно из полей передаваемого объекта сохраняет результат. Вторая проблема решается приведением типов (что показано в коде).

В методе `Main` инициализируем объекты-аргументы и объекты-потоки:

```
static void Main(string[] args)
{
        Data data_1 = new Data { message = "Hello, world! It's the TEST_STRING!!!",
            offset = 95 };
        Data data_2 = new Data { message = "Hello, world! It's the TEST_STRING!!!",
            offset = 12 };
        Thread th_1 = new Thread(EncryptMessage);
        Thread th_2 = new Thread(EncryptMessage);
        th_1.Start(data_1);
        Thread.Sleep(1000);
        th_2.Start(data_2);
        Console.ReadKey();
}
```

Для тестирования были определены сразу два потока.

Итоговый код в листинге ниже:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace thread_args
{
        public struct Data
        {
                public string message;
                public int offset;
        }
        internal class Program
        {
                static void Main(string[] args)
```

```
18              {
19                      Data data_1 = new Data { message = "Hello, world! It's the
                            TEST_STRING!!!", offset = 95 };
20                      Data data_2 = new Data { message = "Hello, world! It's the
                            TEST_STRING!!!", offset = 12 };
21                      Thread th_1 = new Thread(EncryptMessage);
22                      Thread th_2 = new Thread(EncryptMessage);
23                      th_1.Start(data_1);
24                      Thread.Sleep(1000);
25                      th_2.Start(data_2);
26                      Console.ReadKey();
27              }
28
29              static void EncryptMessage(object o)
30              {
31                      string message = ((Data)o).message;
32                      int n = ((Data)o).offset;
33                      StringBuilder sb = new StringBuilder();
34                      foreach (char c in message)
35                      {
36                              char current_c = (char)(c + n);
37                              if(current_c > (char)126)
38                              {
39                                      current_c = (char)(current_c % 127 + ' ');
40                              }
41                              sb.Append(current_c);
42                      }
43                      Console.WriteLine("Исходная строка: {0}", message);
44                      Console.WriteLine("Зашифрованная строка: {0}", sb);
45              }
46      }
47  }
```

Тестирование

Запуск программы осуществим командой, которая скомпилирует проект и запустит его в среде выполнения:

```
1  dotnet build; dotnet run
```

Вывод программы:

```
Исходная строка: Hello, world! It's the TEST_STRING!!!
Зашифрованная строка: Hello, world! It's the TEST_STRING!!!
Исходная строка: Hello, world! It's the TEST_STRING!!!
Зашифрованная строка: Tqxx{8,${ xp-,U!3 ,!tq,`Q_`k_`ÛZS---
```

Программа выполняется корректно. В первом выводе зашифрованная строка эквивалентна исходной из-за циклического сдвига (смещение на 95 символов

в ASCII). Во втором выводе зашифрованная строка соответствует циклическому сдвигу на 12 символов.

Вопросы для защиты работы

1. **Опишите возможные пути передачи параметров в поток.**

   Есть несколько способов передачи параметров в поток:

   - Через аргумент `ParameterizedThreadStart`
   - Через реализацию класса с определением необходимых для работы потока полей. Метод, выполняемый в потоке, будет иметь к ним доступ как член класса.

2. **Подумайте, можно ли передать в несколько потоков один и тот же параметр (ссылку на объект)? Ответ обоснуйте.** Можно, но с оговорками. Если:

   - Объект содержит только `readonly` поля
   - Объект является потокобезопасным (имеет примитивы синхронизации, которые исключают несогласованность работы с полями)

   то такой объект можно передавать в несколько потоков. Иначе - нет.

Выводы

В данной лабораторной работе изучены и продемонстрированы в виде программного кода способы передачи параметров в потоки.